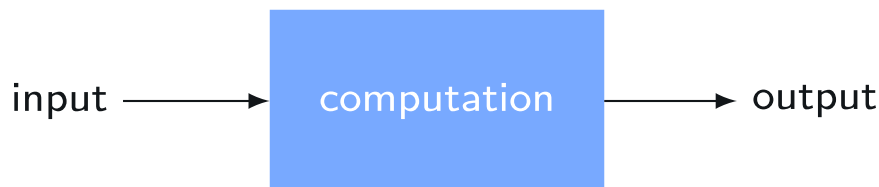


Measuring computational cost

Next, we'll discuss a mathematical framework through which computational cost can be measured, narrowly focused on the needs of this course. The *analysis of algorithms* and *computational complexity* are entire subjects onto themselves, and have much more to say about these notions.

As a starting point, consider the following figure from the previous lesson, which represents a very high level abstraction of a computation.



The computation itself could be modeled or described in a variety of ways, such as by a computer program written in Python, a Turing machine, a Boolean circuit, or a quantum circuit. Our focus will be on circuits (both Boolean and quantum).

Encodings and input length

Let's begin with the input and output of a computational problem, which we'll assume are *binary strings*. Different symbols could be used, but we'll keep things simple for the purposes of this discussion by restricting our attention to binary string inputs and outputs. Through binary strings, we can *encode* a variety of interesting objects that the problems we're interested in solving might concern, such as numbers, vectors, matrices, and graphs, as well as lists of these and other objects.

For example, to encode nonnegative integers, we can use *binary notation*. The following table lists the binary encoding of the first nine nonnegative integers, along with the *length* (meaning the total number of bits) of each encoding.

Number	Binary encoding	Length
0	0	1
1	1	1
2	10	2
3	11	2
4	100	3
5	101	3
6	110	3
7	111	3
8	1000	4

We can easily extend this encoding to handle both positive and negative integers by appending a *sign bit* to the representations if we choose. Sometimes it's also convenient to allow binary representations of nonnegative integers to have leading zeros, which don't change the value being encoded but can allow representations to fill up a string or word of a fixed size.

Using binary notation to represent nonnegative integers is both common and efficient, but if we wanted to we could choose a different way to represent nonnegative integers using binary strings, such as the ones suggested in the following table. The specifics of these alternatives are not important to this discussion — the point is only to clarify that we do have choices for the encodings we use.

Number	Unary encoding	Lexicographic encoding
0	ϵ	ϵ
1	0	0
2	00	1
3	000	00
4	0000	01
5	00000	10
6	000000	11
7	0000000	000
8	00000000	001

(In this table, the symbol ϵ represents the *empty string*, which has no symbols in it and length equal to zero. Naturally, to avoid an obvious source of confusion, we use a special symbol such as ϵ to represent the empty string rather than literally writing nothing.)

Other types of inputs, such as vectors and matrices, or more complicated objects like descriptions of molecules, can also be encoded as binary strings. Just like we have for nonnegative integers, a variety of different encoding schemes can be selected or invented. For whatever scheme we come up with to encode inputs to a given problem, we interpret the *length* of an input string as representing the size of the problem instance being solved.

For example, the number of bits required to express a nonnegative integer N in binary notation, which is sometimes denoted $\lg(N)$, is given by the following formula.

$$\lg(N) = \begin{cases} 1 & N = 0 \\ 1 + \lfloor \log_2(N) \rfloor & N \geq 1 \end{cases}$$

Assuming that we use binary notation to encode the input to the integer factoring problem, the *input length* for the number N is therefore $\lg(N)$. Note, in particular, that the length (or size) of the input N is not N itself; when N is large we don't need nearly this many bits to express N in binary notation.

From a strictly formal viewpoint, whenever we consider a computational problem or task, it should be understood that a specific scheme has been selected for encoding whatever objects are given as input or produced as output. This allows computations that solve interesting problems to be viewed abstractly as transformations from binary string inputs to binary string outputs.

The details of how objects are encoded as binary strings must necessarily be important to these computations at some level. Usually, though, we don't worry all that much about these details when we're analyzing computational cost, so that we can avoid getting into details of secondary importance. The basic reasoning is that we expect the computational cost of converting back and forth between "reasonable" encoding schemes to be insignificant compared with the cost of solving the actual problem. In those situations in which this is not the case, the details can (and should) be clarified.

For example, a very simple computation converts between the binary representation of a nonnegative integer and its lexicographic encoding (which we have not explained in detail, but it can be inferred from the table above). For this reason, the computational cost of integer factoring wouldn't differ significantly if we decided to switch from using one of these encodings to the other for the input N . On the other hand, encoding nonnegative integers in unary notation incurs an exponential blow-up in the total number of symbols required, and we would not consider it to be a "reasonable" encoding scheme for this reason.

Elementary operations

Now let's consider the computation itself, which is represented by the blue rectangle in the figure above. The way that we'll measure computational cost is to count the number of *elementary operations* that each computation requires. Intuitively speaking, an elementary operation is one involving a small, fixed number of bits or qubits, that can be performed quickly and

easily — such as computing the AND of two bits. In contrast, running the `factorint` function is not reasonably viewed as being an elementary operation.

Formally speaking, there are different choices for what constitutes an elementary operation depending on the computational model being used. Our focus will be on circuit models, and specifically quantum and Boolean circuits.

Universal gate sets

For circuit-based models of computation, it's typical that each *gate* is viewed as an elementary operation. This leads to the question of precisely which gates we permit in our circuits. Focusing for the moment on quantum circuits, we've seen several gates thus far in this series, including X , Y , Z , H , S , and T gates, *swap* gates, controlled versions of gates (including *controlled-NOT*, *Toffoli*, and *Fredkin* gates), as well as gates that represent standard basis measurements. In the context of the CHSH game we also saw a few additional *rotation* gates.

We also discussed *query gates* in the context of the query model, and we also saw that any unitary operation U , acting on any number of qubits, can be viewed as being a gate if we so choose — but we'll disregard both of these options for the sake of this discussion. We won't be working in the query model (although the implementation of query gates using elementary operations is discussed later in the lesson), and viewing arbitrary unitary gates, potentially acting on millions of qubits, as being elementary operations does not lead to meaningful or realistic notions of computational cost.

Sticking with quantum gates that operate on small numbers of qubits, one approach to deciding which ones are to be considered elementary is to tease out a precise criterion — but this is not the standard approach or the one we'll take. Rather, we simply make a choice.

Here's one standard choice, which we shall adopt as the *default* gate set for quantum circuits:

- Single-qubit unitary gates from this list: X , Y , Z , H , S , $S^\dagger$, T , and $T^\dagger$
- Controlled-NOT gates

- Single-qubit standard basis measurements

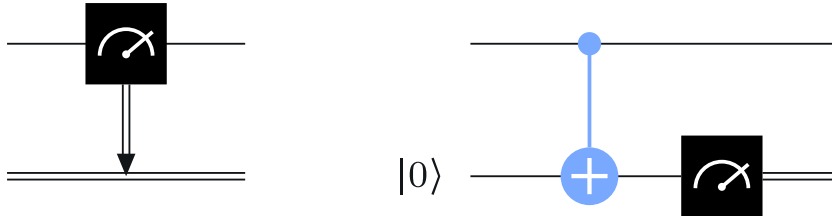
A common alternative is to view Toffoli, Hadamard, and S gates as being elementary, in addition to standard basis measurements. Sometimes all single-qubit gates are viewed as being elementary, though this does lead to an unrealistically powerful model when the accuracy with which gates are performed is not properly taken into account.

The unitary gates in our default collection form what's called a *universal* gate set. This means that we can approximate any unitary operation on any number of qubits to any degree of accuracy we wish, using circuits composed of these gates alone. To be clear, the definition of universality places no requirements on the cost of such approximations, meaning the number of gates from our set that we need. Indeed, a fairly simple argument based on the mathematical notion of measure reveals that most unitary operations must have extremely high cost. Proving the universality of quantum gate sets is not a simple matter and won't be covered in this course.

For Boolean circuits, we'll take AND, OR, NOT, and FANOUT gates to be the ones representing elementary operations. We don't actually need both AND gates and OR gates — we can use *De Morgan's laws* to convert from either one to the other by placing NOT gates on all three input/output wires — but nevertheless it is both typical and convenient to allow both AND and OR gates. AND, OR, NOT, and FANOUT gates form a universal set for deterministic computations, meaning that any function from any fixed number of input bits to any fixed number of output bits can be implemented with these gates.

The principle of deferred measurement

Standard basis measurement gates can appear within quantum circuits, but sometimes it's convenient to delay them until the end. This allows us to view quantum computations as consisting of a unitary part (representing the computation itself), followed by a simple read-out phase where qubits are measured and the results are output. This can always be done, provided that we're willing to add an additional qubit for each standard basis measurement. In the following figure, the circuit on the right illustrates how this can be done for the gate on the left.



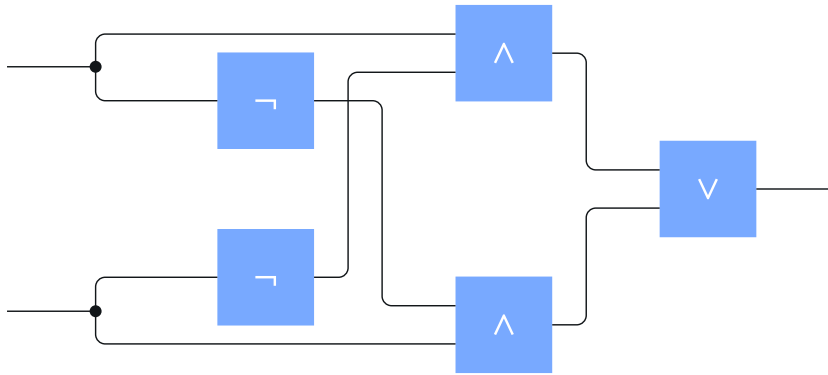
Specifically, the classical bit in the circuit on the left is replaced by a qubit on the right (initialized to the $|0\rangle$ state), and the standard basis measurement is replaced by a controlled-NOT gate, followed by a standard basis measurement on the bottom qubit. The point is that the standard basis measurement in the right-hand circuit can be pushed all the way to the end of the circuit. If the classical bit in the circuit on the left is later used as a control bit, we can use the bottom qubit in the circuit on the right as a control instead, and the overall effect will be the same. (We are assuming that the classical bit in the circuit on the left doesn't get overwritten after the measurement takes place by another measurement — but if it did we could always just use a new classical bit rather than overwriting one that was used for a previous measurement.)

Circuit size and depth

Circuit size

The total number of gates in a circuit is referred to as that circuit's *size*. Thus, presuming that the gates in our circuits represent elementary operations, a circuit's size represents the number of elementary operations it requires — or, in other words, its *computational cost*. We write $\text{size}(C)$ to refer to the size of a given circuit C .

For example, consider the following Boolean circuit for computing the XOR of two bits.



The size of this circuit is 7 because there are 7 gates in total. (Fanout operations are not always counted as being gates, but for the purposes of this lesson we will count them as being gates.)

Time, cost, and circuit depth

Time is a critically important resource, or a limiting constraint, for computations. The examples above, such as the task of factoring RSA1024, reinforce this viewpoint. The `factorint` function doesn't fail to factor RSA1024 per se, it's just that we don't have enough time to let it finish.

The notion of computational cost, as the number of elementary operations required to perform a computation, is intended to be an abstract proxy for the time required to implement a computation. Each elementary operation requires a certain amount of time to perform, and the more of them a computation needs, the longer it's going to take, at least in general. In the interest of simplicity, we'll continue to make this association between computational cost and the time required to run algorithms.

But notice that the size of a circuit doesn't necessarily correspond directly to how long it takes to run. In our Boolean circuit for computing the XOR of two bits, for instance, the two FANOUT gates could be performed simultaneously, as could the two NOT gates, as well as the two AND gates. A different way to measure the efficiency of circuits, which takes this possibility of *parallelization* into account, is by their *depth*. This is the minimum number of *layers* of gates needed within the circuit, where the gates within each layer operate on different wires. Equivalently, the depth of a circuit is the maximum number of gates encountered on any path from an input wire to an output wire. For the circuit above, for instance, the depth is

4.

Circuit depth is one way to formalize the running time of parallel computations. It's an advanced topic, and there exist very sophisticated circuit constructions known to minimize the depth required for certain computations. There are also some fascinating unanswered questions concerning circuit depth. (For example, much remains unknown about the circuit depth required to compute GCDs.) We won't have too much more to say about circuit depth in this series, aside from including a few interesting facts concerning circuit depth as we go along, but it should be clearly acknowledged that parallelization is a potential source of computational advantages.

Assigning costs to different gates

One final note concerning circuit size and computational cost is that it is possible to assign different costs to gates, rather than viewing every gate as contributing equally to the total cost.

For example, as was already mentioned, FANOUT gates are often viewed as being free for Boolean circuits — which is to say that we could choose that FANOUT gates have zero cost. As another example, when we're working in the query model and we count the number of queries that a circuit makes to an input function (in the form of a black box), we're effectively assigning unit cost to query gates and zero cost to other gates, such as Hadamard gates. A final example is that we sometimes assign different costs to gates depending on how difficult they are to implement, which could vary depending upon the hardware being considered.

While all of these options are sensible in different contexts, for this lesson we'll keep things simple and stick with circuit size as a representation of computational cost.

Cost as a function of input length

We're primarily interested in how computational cost scales as inputs become larger and larger. This leads us to represent the costs of algorithms as *functions* of the input length.

Families of circuits

Inputs to a given computational problem can vary in length, potentially becoming arbitrarily large. Every circuit, on the other hand, has a fixed number of gates and wires. For this reason, when we think about algorithms in terms of circuits, we generally need infinitely large *families* of circuits to represent algorithms. By a family of circuits, we mean a sequence of circuits that grow in size, allowing larger and larger inputs to be accommodated.

For example, imagine that we have a classical algorithm for integer factorization, such as the one used by `factorint`. Like all classical algorithms, this algorithm can be implemented using Boolean circuits — but to do it we'll need a separate circuit for each possible input length. If we looked at the resulting circuits for different input lengths, we would see that their sizes naturally grow as the input length grows — reflecting the fact that factoring 4-bit integers is much easier and requires far fewer elementary operations than factoring 1024-bit integers, for instance.

This leads us to represent the computational cost of an algorithm by a function t , defined so that $t(n)$ is the number of gates in the circuit that implements the algorithm for n bit inputs. In more formal terms, an algorithm in the Boolean circuit model is described by a sequence of circuits $\{C_1, C_2, C_3, \dots\}$, where C_n solves whatever problem we're talking about for n -bit inputs (or, more generally, for inputs whose size is parameterized in some way by n), and the function t representing the cost of this algorithm is defined as

$$t(n) = \text{size}(C_n).$$

For quantum circuits the situation is similar, where larger and larger circuits are needed to accommodate longer and longer input strings.

Example: integer addition

To explain further, let's take a moment to consider the problem of integer addition, which is much simpler than integer factoring or even computing GCDs.

Integer addition

Input: integers N and M

Output: $N + M$

How might we design Boolean circuits for solving this problem?

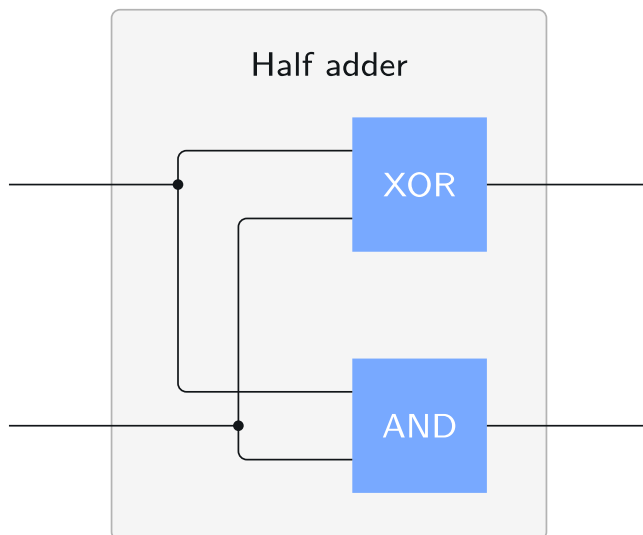
To keep things simple, let's restrict our attention to the case where N and M are both nonnegative integers represented by n -bit strings using binary notation. We'll allow for any number of leading zeros in these encodings, so that

$$0 \leq N, M \leq 2^n - 1.$$

The output will be an $(n + 1)$ -bit binary string representing the sum, which is the maximum number of bits we need to express the result.

We begin with an algorithm — the *standard* algorithm for addition of binary representations — which is the base 2 analogue to the way addition is taught in elementary/primary schools around the world. This algorithm can be implemented with Boolean circuits as follows.

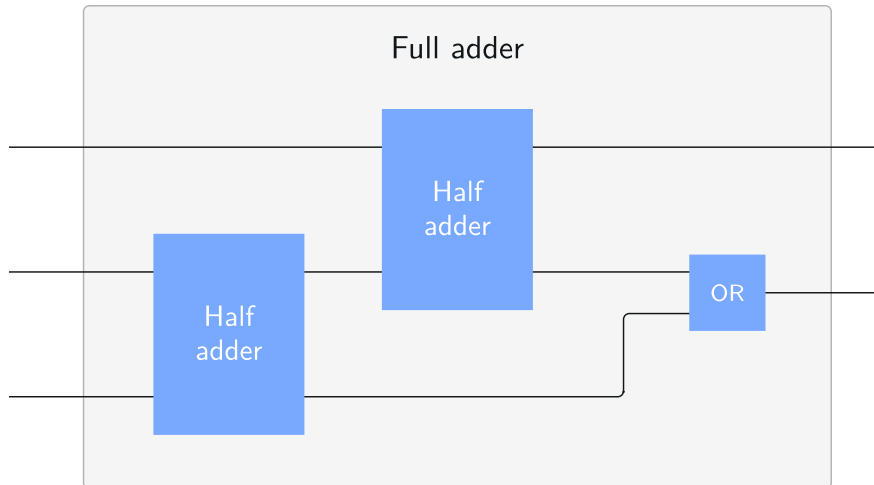
Starting from the least significant bits, we can compute their XOR to determine the least significant bit for the sum. Then we compute the carry bit, which is the AND of the two least significant bits of N and M . Sometimes these two operations together are known as a *half adder*.



Using the XOR circuit we've now seen a few times together with an AND gate and two FANOUT gates, we can build a half adder with 10 gates. If for some reason we changed our minds and decided to include XOR gates in our set of

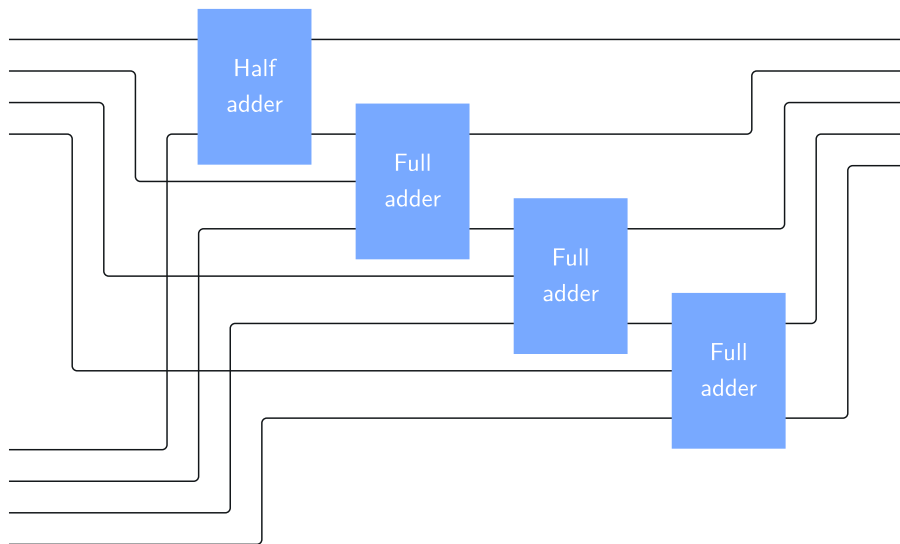
elementary operations, we would need 1 AND gate, 1 XOR gate, and 2 FANOUT gates to build a half adder.

Moving on to the more significant bits, we can use a similar procedure, but this time including the carry bit from each previous position into our calculation. By cascading two half adders and taking the OR of the carry bits they produce, we can create what's known as a *full adder*.



This requires 21 gates in total: 2 AND gates, 2 XOR gates (each requiring 7 gates to implement), one OR gate, and 4 FANOUT gates.

Finally, by cascading the full adders, we obtain a Boolean circuit for nonnegative integer addition. For example, the following circuit computes the sum of two 4-bit integers.



In general, this requires

$$21(n - 1) + 10 = 21n - 11$$

gates. Had we decided to include XOR gates in our set of elementary operations, we would need $2n - 1$ AND gates, $2n - 1$ XOR gates, $n - 1$ OR gates, and $4n - 2$ FANOUT gates, for a total of $9n - 5$ gates. If in addition we decide not to count FANOUT gates, it's $5n - 3$ gates.

Asymptotic notation

On the one hand, it's good to know precisely how many gates are needed to perform various computations, like in the example of integer addition above. These details are important for actually building the circuits.

On the other hand, if we perform analyses at this level of detail for all the computations we're interested in, including ones for tasks that are much more complicated than addition, we'll very quickly be buried in details. To keep things manageable, and to intentionally suppress details of secondary importance, we typically use *Big-O* notation when analyzing algorithms. Through this notation we can express the *order* at which functions grow.

Formally speaking, if we have two functions $g(n)$ and $h(n)$, we write that $g(n) = O(h(n))$ if there exists a positive real number $c > 0$ and a positive integer n_0 such that

$$g(n) \leq c \cdot h(n)$$

for all $n \geq n_0$. Typically $h(n)$ is chosen to be as simple an expression as possible, so that the notation can be used to reveal the limiting behavior of a function in simple terms. For example, $17n^3 - 257n^2 + 65537 = O(n^3)$.

This notation can be extended to functions having multiple arguments in a fairly straightforward way. For instance, if we have two functions $g(n, m)$ and $h(n, m)$ defined on positive integers n and m , we write that $g(n, m) = O(h(n, m))$ if there exists a positive real number $c > 0$ and a positive integer k_0 such that

$$g(n, m) \leq c \cdot h(n, m)$$

whenever $n + m \geq k_0$.

Connecting this notation to the example of nonnegative integer addition, we conclude that there exists a family of Boolean circuits $\{C_1, C_2, \dots\}$,

where C_n adds two n -bit nonnegative integers together, such that $\text{size}(C_n) = O(n)$. This reveals the most essential feature of how the cost of addition scales with the input size: it scales *linearly*.

Notice also that it doesn't depend on the specific detail of whether we consider XOR gates to have unit cost or cost 7. In general, using Big-O notation allows us to make statements about computational costs that aren't sensitive to such low-level details.

More examples

Here are a few more examples of problems from computational number theory, beginning with *multiplication*.

Integer multiplication

Input: integers N and M

Output: NM

Creating Boolean circuits for this problem is more difficult than creating circuits for addition — but by thinking about the standard multiplication algorithm, we can come up with circuits having size $O(n^2)$ for this problem (assuming N and M are both represented by n -bit binary representations). More generally, if N has n bits and M has m bits, there are Boolean circuits of size $O(nm)$ for multiplying N and M .

There are, in fact, other ways to multiply that scale better. For instance, the Schönhage-Strassen multiplication algorithm can be used to create Boolean circuits for multiplying two n -bit integers at cost $O(n \lg(n) \lg(\lg(n)))$. The intricacy of this method causes a lot of overhead, however, making it only practical for numbers having tens of thousands of bits.

Another basic problem is *division*, which we interpret to mean computing both the quotient and remainder given an integer divisor and dividend.

Integer division

Input: integers N and $M \neq 0$

Output: integers q and r satisfying $0 \leq r < |M|$ and $N = qM + r$

The cost of integer division is similar to multiplication: if N has n bits and M has m bits, there are Boolean circuits of size $O(nm)$ for solving this

problem. And like multiplication, asymptotically superior methods are known.

We can now compare known algorithms for computing GCDs with those for addition and multiplication. Euclid's algorithm for computing the GCD of an n -bit number N and an m -bit number M requires Boolean circuits of size $O(nm)$, similar to the standard algorithms for multiplication and division. Also similar to multiplication and division, there are asymptotically faster GCD algorithms — including ones requiring $O(n(\lg(n))^2 \lg(\lg(n)))$ elementary operations to compute the GCD of two n -bit numbers.

A somewhat more expensive computation that arises in number theory is *modular exponentiation*.

Integer modular exponentiation

Input: integers N , K , and M with $K \geq 0$ and $M \geq 1$

Output: $N^K \pmod{M}$

By $N^K \pmod{M}$ we mean the remainder when N^K is divided by M , meaning the unique integer r satisfying $0 \leq r < M$ and $N^K = qM + r$ for some integer q .

If N has n bits, M has m bits, and K has k bits, this problem can be solved by Boolean circuits having size $O(km^2 + nm)$. This is not at all obvious. The solution is not to first compute N^K and then take the remainder, which would necessitate using exponentially many bits just to store the number N^K . Rather, we can use the *power algorithm* (known alternatively as the *binary method* and *repeated squaring*), which makes use of the binary representation of K to perform the entire computation modulo M . Assuming N , M , and K are all n -bit numbers, we obtain an $O(n^3)$ algorithm — or a *cubic* time algorithm. And once again, there are known algorithms that are more complicated but asymptotically faster.

Cost of integer factorization

In contrast to the algorithms just discussed, known algorithms for integer factorization are much more expensive — as we might expect from the discussion earlier in the lesson.

One simple approach to factoring is *trial division*, where an algorithm searches through the list $2, \dots, \sqrt{N}$ to find a prime factor of an input number N . This requires $O(2^{n/2})$ iterations in the worst case when N is an n -bit number. Each iteration requires a trial division, which means $O(n^2)$ elementary operations for each iteration (using a standard algorithm for integer division). We end up with circuits of size $O(n^2 2^{n/2})$, which is *exponential* in the input size n .

There are algorithms for integer factorization having better scaling. The number field sieve mentioned earlier, for instance, which is an algorithm that makes use of randomness, is generally believed (but not rigorously proven) to require

$$2^{O(n^{1/3}(\lg(n))^{2/3})}$$

elementary operations to factor n -bit integers with high probability. While it is quite significant that n is raised to the power $1/3$ in the exponent of this expression, the fact it appears in the exponent is still a problem that causes poor scaling — and explains in part why RSA1024 remains outside of its domain of applicability.

Polynomial versus exponential cost

Classical algorithms for integer addition, multiplication, division, and computing greatest common divisors allow us to solve these problems in the blink of an eye for inputs with thousands of bits. Addition has *linear* cost while the other three problems have *quadratic* cost (or *subquadratic* cost using asymptotically fast algorithms). Modular exponentiation is more expensive but can still be done pretty efficiently, with *cubic* cost (or sub-cubic cost using asymptotically fast algorithms).

These are all examples of algorithms having *polynomial* cost, meaning that they have cost $O(n^c)$ for some choice of a fixed constant $c > 0$. As a rough, first-order approximation, algorithms having polynomial cost are abstractly viewed as representing *efficient* algorithms.

In contrast, known classical algorithms for integer factoring have *exponential* cost. Sometimes the cost of the number field sieve is described as *sub-exponential* because n is raised to the power $1/3$ in the exponent, but in complexity theory it is more typical to reserve this term for algorithms whose cost is

$$O(2^{n^\epsilon})$$

for every $\epsilon > 0$. The so-called *NP-complete* problems are a class of problems not known to (and widely conjectured not to) have polynomial-cost algorithms. A circuit-based formulation of the *exponential-time hypothesis* posits something even stronger, which is that no NP-complete problem can have a sub-exponential cost algorithm.

The association of polynomial-cost algorithms with efficient algorithms must be understood as being a loose abstraction. Of course, if an algorithm's cost scales as n^{1000} or $n^{1000000}$ for inputs of size n , then it's a stretch to describe that algorithm as being efficient. However, even an algorithm having cost that scales as $n^{1000000}$ must be doing something clever to avoid having *exponential* cost, which is generally what we expect of algorithms based in some way on "brute force" or "exhaustive search." Even the sophisticated refinements of the number field sieve, for instance, fail to avoid this exponential scaling in cost. Polynomial-cost algorithms, on the other hand, manage to take advantage of the problem structure in some way that avoids an exponential scaling.

In practice, the identification of a polynomial-cost algorithm for a problem is just a first step toward actual efficiency. Through algorithmic refinements, polynomial-cost algorithms with large exponents can sometimes be improved dramatically, lowering the cost to a more "reasonable" polynomial scaling. Sometimes things become easier when they're known to be possible — so the identification of a polynomial-cost algorithm for a problem can also have the effect of inspiring new, even more efficient algorithms.

As we consider advantages of quantum computing over classical computing, our eyes are generally turned first toward *exponential* advantages, or at least *super-polynomial* advantages — ideally finding polynomial-cost quantum algorithms for problems not known to have polynomial-cost classical algorithms. Theoretical advantages on this order have the greatest chances to lead to actual practical advantages — but identifying such advantages is an extremely difficult challenge. Only a few examples are currently known, but the search continues.

Polynomial (but not super-polynomial) advantages in computational cost of quantum over classical are also interesting and should not be dismissed — but given the current gap between quantum and classical computing technology, they do seem rather less compelling at the present time. One day, though, they could become significant. *Grover's algorithm*, for instance, which is covered in a later lesson, offers a *quadratic* advantage of quantum

over classical for so-called *unstructured searching*, and has a potential for broad applications.

A hidden cost of circuit computation

There is one final issue that's worth mentioning, although we will not concern ourselves with it further in this course. There's a "hidden" computational cost when we're working with circuits, and it concerns the specifications of the circuits themselves. As inputs get longer and longer, larger and larger circuits are required — but we need to get our hands on the descriptions of these circuits somehow if we're going to implement them.

For all of the examples we've discussed, or will discuss in subsequent lessons, there's an underlying algorithm from which the circuits are derived. Usually the circuits in a family follow some basic pattern that's easy to extrapolate to larger and larger inputs, such as cascading full adders to create Boolean circuits for addition or performing layers of Hadamard gates and other gates in some simple to describe pattern.

But what happens if there are prohibitive computational costs associated with the patterns in the circuits themselves? For instance, the description of each member C_n in a circuit family could, in principle, be determined by some extremely difficult to compute function of n .

The answer is that this is indeed a problem — and so we must place additional restrictions on families of circuits beyond having polynomial cost in order for them to truly represent efficient algorithms. The property of *uniformity* for circuits does this by stipulating that, in various precise formulations, it must be computationally easy to obtain the description of each circuit in a family. All of the circuit families we'll discuss do have this property — but this is nevertheless an important issue to be aware of in general when studying circuit models of computation from a formal viewpoint.

Was this page helpful?

Yes 	No 
---	--

Report a bug, typo, or request content on [GitHub](#).

[Previous page](#)

[Next page](#)

© IBM Corp., 2017-2026